

DOCUMENT 1 OF 4

# CLAUDE.md — Development Guide

## Luka — Claude Development Guide

---

Luka is a Latin American personal finance SaaS for individuals, couples, and groups of people. It automatically captures transactions from bank email notifications (Gmail/Outlook), bank scraping (Luka Connect), and open banking (Plaid for US); classifies them via LLM; enables actions via WhatsApp; and visualizes spending on a responsive web dashboard. The platform is LATAM-first — supporting CL, CO, MX, PE, BR, and US — with a three-layer email parser (declarative templates → Gemini LLM waterfall → legacy regex) covering 101 banks across 6 countries.

---

### Problem to be Solved

---

- **User:** Difficulty managing personal finances (expenditure, budgeting, income, etc.) across multiple bank accounts, fintech accounts, cards, shared/household expenses, subscriptions, and currencies.
  - **Customer (B2B):** Low to no access to good-quality financial behavior data from Latin American individuals for fintechs and financial institutions to assess and offer tailored, profitable financial products.
- 

### Target User

---

Latin American mid-to-high income banked or semi-banked individuals with any level of financial knowledge, wanting to track their expenses and optimize their personal finance management.

---

### Value-add to the User

---

1. One-stop automated platform (web + app) to smoothly track, understand, analyze, and act on all personal finances.

2. AI-generated classification and automatic notification of every income and expense transaction — live, not on a weekly or monthly basis.
  3. Single application combining what users currently do across multiple apps (e.g., Splitwise, Rocket Money, traditional bank apps) — not necessarily replacing them, but connecting and displaying everything in one place.
  4. Predictive budgeting and personalized best practices to achieve specific financial goals, based on the user's own behavior.
- 

## Target Customer

---

- Latin American fintechs and financial institutions looking to provide credit and offer financial products to creditworthy individuals.
  - Eventually, other institutions wanting to understand individual financial behavior to offer products or services (e.g., consumer goods, travel and leisure companies).
- 

## Value-add to the Customer

---

1. Structured and detailed individual financial behavior data (aggregated or individual) to make creditworthiness predictions and offer personalized financial products to current or new customers.
  2. Centralized channel to access a pool of creditworthy individuals who previously had no relationship with their institution.
- 

## Business Model

---

Two-sided revenue stream:

- **Users** pay a small subscription to use the app with their data kept private.
- **B2B customers** pay for aggregated, anonymized financial behavior data — similar to YouTube's ad model where users can pay to opt out.

The user always comes first. A large, happy user base generating behavioral data is the foundation that makes the B2B product valuable. Never compromise user trust or experience for short-term B2B gain.

The model can evolve. If during development you spot a new revenue opportunity or a structural improvement, flag it.

---

# Tech Stack (summary)

| Layer          | Stack                                                                                     |
|----------------|-------------------------------------------------------------------------------------------|
| Backend        | Python 3.12 + FastAPI 0.111, SQLAlchemy 2.0 async, ARQ queue, Redis                       |
| Frontend       | Next.js 16 (App Router), React 19, Tailwind CSS 4, shadcn/ui, Zustand 5, TanStack Query 5 |
| Database       | Supabase PostgreSQL 15, Alembic migrations                                                |
| Auth           | Supabase Auth (Google/Microsoft OAuth), PyJWT + JWKS                                      |
| LLM            | Google Gemini (email parsing waterfall + merchant categorization)                         |
| Infrastructure | Railway (backend + workers), Vercel (frontend), Google Cloud Pub/Sub                      |

**Scope:** LATAM-first — currently supporting CL, CO, MX, PE, BR, and US. Bank email parsing, currencies, and transaction formats must account for all supported countries. Nothing should be hardcoded for Chile only.

See [ARCHITECTURE.md](#) for the full data model, API endpoints, and key flows.

## Coding Values

1. **User first.** Every coding and product decision must center on the user — making it as simple, clear, and useful as possible. Never degrade the user experience.
2. **Balance complexity and usability.** More is not always better. Keep the user experience as simple as possible.
3. **Be diligent.** Always achieve the best possible outcome given time and resource constraints.
4. **Don't assume — ask.** If an instruction lacks a clear goal or a clear path, ask clarifying questions before starting.
5. **Challenge the status quo.** Always use state-of-the-art tools and techniques. Research what "state of the art" looks like before defaulting to the familiar.
6. **Brevity over cleverness.** Short, clear code. No unnecessary abstractions or bells and whistles.

# Ways of Working

---

## Who we are

Two Wharton MBAs (mechanical engineer + industrial engineer). We can code and review errors quickly, and have strong mathematical backgrounds for verifying logic. Our focus is product and business — not reviewing every line of code.

## Workflow rules

- **Before designing a new feature:** use `/brainstorming` to align on approach first.
- **After every significant code change:** run `requesting-code-review` to verify efficiency and correctness.
- **After every feature implementation:** run a verification pass — test locally or use `/browser-use` for frontend checks. Target: 95%+ of proposed implementation working before marking complete.
- **For final polish:** use iteration loops to catch edge cases and regressions.
- **For all frontend work:** maintain UI/UX consistency across the app. Mobile-first, clean, and coherent with existing design language.

## Documentation — keep these always up to date

- `README.md` — project overview and feature tracking.
- `ARCHITECTURE.md` — tech stack, data model, API endpoints, key flows.
- `NEXT-STEPS.md` — pending features, known issues, future ideas.
- `CLAUDE.md` — update this file whenever a major architectural or product decision is made.

## Project hygiene

- Keep the directory clean. Delete every folder and file that will no longer be used.
- Prefer editing existing files over creating new ones.
- No orphan files, dead code, or half-finished implementations left behind.

---

## Key Architectural Conventions

---

- **Amount sign convention:** expenses and transfers stored as negative, income as positive — everywhere (email, Plaid, bank connect).
- **Transaction types:** `expense` / `income` / `transfer`. "Transfer" = own-account moves only (CC bill payments, checking → savings). Person-to-person payments (Zelle, etc.) = expense/income.

- **Split types:** `personal` / `shared`. Joint bank accounts auto-classify all transactions as shared.
  - **Multi-currency:** transactions carry their own currency. Never hardcode USD or assume a single currency.
  - **Email parsing layers:** Template (zero LLM cost) → Gemini waterfall (4 models, confidence-based) → legacy regex fallback.
  - **Worker routing:** fast worker (email, cron,  $\leq 60s$ ) vs. slow worker (bank syncs, LLM review,  $\leq 600s$ ).
  - **Categories:** fetched dynamically via API. All dropdowns reflect user preferences in real time.
  - **Testing:** backend uses pytest with `asyncio_mode = auto`. No DB mocks — tests hit a real database. Frontend has no test infrastructure yet.
-

DOCUMENT 2 OF 4

# README.md — Project Overview

## Luka

---

Latin American personal finance SaaS for individuals, couples, and groups. Automatically captures transactions from bank email notifications (Gmail/Outlook), bank scraping (Luka Connect), and open banking (Plaid for US); classifies them via LLM; enables actions via WhatsApp; and visualizes spending on a responsive web dashboard.

**LATAM-first:** Three-layer email parser (declarative templates, Gemini LLM waterfall, legacy regex) covering 101 banks across 6 countries (CL, CO, MX, PE, BR, US). Multi-currency throughout — nothing hardcoded to a single country or currency.

## Tech Stack

---

### Backend

- Python 3.12 + FastAPI 0.111
- SQLAlchemy 2.0 async + asyncpg (PostgreSQL ORM)
- ARQ async job queue (two-tier: fast + slow workers)
- Redis (caching + job queue)
- Google Gemini (LLM email parsing + merchant categorization)

### Frontend

- Next.js 16 (App Router) + React 19
- Tailwind CSS 4 + shadcn/ui (Radix-based components)
- Zustand 5 (client state, persisted to localStorage)
- TanStack Query 5 (server state, 30s staleTime)
- Recharts (financial charts)

### Database

- Supabase PostgreSQL 15
- Alembic migrations (37 versions)

### Auth

- Supabase Auth — Google OAuth (Gmail users) + Microsoft OAuth (Outlook users)
- PyJWT + JWKS validation (ES256/RS256/HS256 fallback chain)

## Infrastructure

- Railway (backend API + fast worker + slow worker)
- Vercel (frontend)
- Google Cloud Pub/Sub (Gmail push notifications)
- Resend (transactional emails)

# Getting Started

---

## Prerequisites

- Python 3.12+
- Node.js 18+
- PostgreSQL (or Supabase project)
- Redis

## Installation

```
# Backend
cd backend
cp .env.example .env          # Fill in required env vars (see below)
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
alembic upgrade head

# Frontend
cd frontend
cp .env.local.example .env.local  # Set NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_API_URL
npm install
```

## Running Locally

```
# Backend API server
cd backend && uvicorn main:app --reload      # http://localhost:8000

# ARQ workers (separate terminals)
arq worker.FastWorkerSettings              # email processing, light cron
arq worker.SlowWorkerSettings              # bank syncs, LLM, reconciliation, template ager

# Frontend
cd frontend && npm run dev                  # http://localhost:3000
```

## Project Structure

```
backend/
  main.py          FastAPI app factory + 16 routers
  worker.py        ARQ FastWorkerSettings + SlowWorkerSettings
  core/            Config, database, security (PyJWT), cache (Redis)
  modules/         Feature modules (see "What Luka Does" below)
  jobs/            ARQ task definitions + queue routing
  alembic/         Database migrations (37 versions)
  tests/           56 test files (~401 tests)

frontend/
  app/
    (auth)/        Login, onboarding (setup-household, connect-bank, verify-whatsapp)
    (dashboard)/   Home, transactions, budgets, household, subscriptions, notifications,
    (public)/      Privacy policy, terms, data-deletion
    lib/           API client, Zustand store, React Query hooks, Supabase clients

docs/              Architecture docs, design specs, research
```



# Environment Variables

| Variable                                | Required | Description                                                    |
|-----------------------------------------|----------|----------------------------------------------------------------|
| <code>DATABASE_URL</code>               | Yes      | PostgreSQL async connection string                             |
| <code>REDIS_URL</code>                  | Yes      | Redis for caching and job queue                                |
| <code>SUPABASE_URL</code>               | Yes      | Supabase project URL                                           |
| <code>SUPABASE_ANON_KEY</code>          | Yes      | Supabase anonymous key                                         |
| <code>SUPABASE_SERVICE_KEY</code>       | Yes      | Supabase service role key                                      |
| <code>FRONTEND_URL</code>               | Yes      | Frontend URL for CORS and invite links                         |
| <code>GCP_PROJECT_ID</code>             | Yes      | Google Cloud project for Pub/Sub                               |
| <code>PUBSUB_AUDIENCE</code>            | Yes      | Gmail webhook push endpoint URL                                |
| <code>WHATSAPP_APP_SECRET</code>        | Yes      | Meta WhatsApp Cloud API secret                                 |
| <code>WHATSAPP_PHONE_NUMBER_ID</code>   | Yes      | WhatsApp sender phone number ID                                |
| <code>WHATSAPP_ACCESS_TOKEN</code>      | Yes      | WhatsApp Cloud API access token                                |
| <code>GEMINI_API_KEY</code>             | Yes      | Google Gemini for email parsing + merchant categorization      |
| <code>OPENAI_API_KEY</code>             | No       | OpenAI fallback for categorization                             |
| <code>PLAID_CLIENT_ID</code>            | No       | Plaid client ID (US bank connections)                          |
| <code>PLAID_SECRET</code>               | No       | Plaid API secret                                               |
| <code>PLAID_ENV</code>                  | No       | Plaid environment (sandbox/production)                         |
| <code>MICROSOFT_CLIENT_ID</code>        | No       | Azure app for Outlook integration                              |
| <code>MICROSOFT_CLIENT_SECRET</code>    | No       | Azure app secret                                               |
| <code>OUTLOOK_CLIENT_STATE</code>       | No       | Outlook webhook verification secret                            |
| <code>LLM_WATERFALL_THRESHOLDS</code>   | No       | Confidence thresholds per model (default: 0.9,0.8,0.7,0.0)     |
| <code>LLM_SHADOW_VALIDATION_RATE</code> | No       | Shadow validation sample rate (default: 0.25)                  |
| <code>TEMPLATE_AGENT_MIN_EMAILS</code>  | No       | Min LLM-parsed emails before template generation (default: 20) |

| Variable                 | Required | Description                                                                |
|--------------------------|----------|----------------------------------------------------------------------------|
| <code>ENVIRONMENT</code> | No       | <code>development</code> or <code>production</code> (default: development) |

## What Luka Does

### Auth & Users ( `backend/modules/auth/` )

Handles user registration via Supabase OAuth (Google/Microsoft), profile management, encrypted email provider token storage, and WhatsApp PIN verification. Users are auto-provisioned on first OAuth login.

### Households & Partners ( `backend/modules/households/` )

Supports individual and couple households. Partners can be invited via shareable links. Manages split ratios, contribution tracking, settlement calculations, and category breakdowns between partners. No debt ledger — contributions are computed via aggregate queries.

### Email Pipeline ( `backend/modules/email/` )

Three-layer email parsing system for LATAM expansion:

1. **Layer 1 — Declarative Templates:** Auto-generated JSON extraction templates (CSS selectors + regex + fixed transforms). Zero LLM cost per email once promoted.
2. **Layer 2 — Gemini LLM Waterfall:** Confidence-based model escalation (3.1 Flash Lite -> 2.5 Flash -> 3 Flash -> 2.5 Pro). Circuit breaker for API outages (50% error rate -> 15min cooldown).
3. **Layer 3 — Legacy Regex:** Fallback for Banco de Chile formats with transaction type inference from subject/body keywords.

Receives Gmail push notifications via Google Cloud Pub/Sub and Outlook webhooks via Microsoft Graph. DB-backed bank registry (101 banks across CL, CO, MX, PE, BR, US) with Redis-cached lookups replaces hardcoded sender domains. Financial keyword filter now supports Spanish, English, and Portuguese. Empty email bodies are detected and skipped before parsing to prevent LLM hallucinations.

### Template Agent ( `backend/modules/email/template_agent.py` )

Autonomous ARQ cron job (daily 2am on slow worker). Discovers banks with sufficient LLM-parsed emails but no active template, generates declarative templates via Gemini, validates against LLM ground truth (100% amount match, 95% merchant match required), promotes to production, and auto-retires on shadow validation drift.

### Bank Registry ( `backend/modules/email/bank_registry_service.py` )

DB-backed bank registry with Redis cache (24h TTL). 101 banks seeded across 6 countries. Supports exact domain match + subdomain fallback. Stores known subjects, notification types, and active template references per bank.

### Bank Connect — Chile ( `backend/modules/bank_connect/` )

Integrates with Luka Connect, a standalone bank scraping service (separate repo). Stores AES-256-GCM

encrypted bank credentials, triggers scraping jobs, maps scraped movements to Luka transactions with deduplication. Scheduled syncs run every 6 hours.

### Bank Connect — US ( [backend/modules/plaid/](#) )

Plaid Link integration for US bank accounts (production). Handles link token creation, public token exchange, transaction sync with cursor pagination, and account kind detection. Amounts stored in cents. Smart transaction mapping: extracts person names from Zelle transfers, detects credit card payments (Amex, Chase, etc.) and only flags as "transfer" if the target card account exists in the system. Person-to-person payments (Zelle, bank transfers) are classified as expense/income, not transfer. Pending (processing) Plaid transactions are surfaced in the frontend pending section with a "bank" badge. Scheduled syncs run daily.

### Transaction Processing ( [backend/modules/transactions/](#) )

Core transaction storage with support for personal, partner, and shared split types. Tracks transaction type (income/expense/transfer), source (email/bank\_connect/plaid/whatsapp/manual), and reconciliation status. Unified sign convention: expenses and transfers are stored as negative amounts, income as positive — consistent across all sources (email, Plaid, bank connect). Optimistic category updates with merchant feedback loops.

### Reconciliation ( [backend/modules/reconciliation/](#) )

Matches email-sourced transactions with bank-sourced transactions using 3-tier priority matching (exact -> fuzzy -> sum-match). Compares absolute amounts to handle sign differences between sources (email=positive, Plaid/connect=negative). Runs during both Plaid sync and luka-connect sync. Enriches bank transactions with user-edited categories from email txs. Also detects transfers between accounts (same-amount opposite-sign pairs).

### Merchant Categorization ( [backend/modules/merchants/](#) , [backend/modules/merchant\\_review/](#) )

Two-tier system: known merchants resolve instantly from a global DB cache, new merchants get 3 LLM-suggested categories (Gemini 2.5 Flash). Canonical merchant grouping via LLM batching. Training UI at [/train](#) for local admin use. User category selections feed back into the merchant database.

### WhatsApp Integration ( [backend/modules/whatsapp/](#) )

Full conversational flow via Meta WhatsApp Cloud API. Transaction alerts include sender, subject, and Chile-timezone time. Interactive split type selection (Personal/Hogar), category picker, and confirmation. Transfers (CC payments, own-account moves) receive an informational-only message with no split/category flow. Supports manual expense entry via natural language.

### Budgets & Allocations ( [backend/modules/budgets/](#) )

Monthly household budgets with waterfall ceiling logic and a multi-level Sankey flow visualization. The v3 [/budgets/v2/{household\\_id}](#) endpoint dispatches to two dedicated builders: [\\_build\\_hogar\\_sankey](#) (4 levels: per-source income → [Ingresos Hogar](#) hub → 5 allocation nodes including [Gastos fijos](#) / [Cuotas](#) / [Meta de ahorro](#) / [Gasto personal](#) / [Disponibile hogar](#) → per-category breakdown) and [\\_build\\_personal\\_sankey](#) (3 allocation levels with its own [Mis ingresos](#) hub). Caller-relative privacy: each viewer sees their own income categories broken out at Level 0, while other members appear as one

aggregated node per member ( `Ingresos {Name}` for full mode, `Contribución fija {Name}` for fixed mode). Forecast engine, 50/20/30 allocation suggestions, per-category caps, cuotas (installment purchases), savings target + payday + personal allocation settings, contribution-mode dispatch (full / fixed / reimbursement) with privacy invariant enforced by construction in `contribution_service.income_breakdown_for_household_view`. Budget configuration (savings target, payday, personal allocation, contribution mode, per-category caps) lives in a single accordion modal on `/budgets` triggered by a gear button next to the currency toggle — no more navigating to `/settings` to tweak a number.

### Notifications ( `backend/modules/notifications/` )

In-app notification system with unread counts, per-user notification preferences (WhatsApp toggle), and CRUD operations.

### Subscriptions ( `backend/modules/subscriptions/` )

Automatic recurring transaction detection via DB-backed cache (refreshed on demand). Each detected subscription supports an explicit user-set classification ( `Personal` or `Compartido` / shared) via a click-to-flip pill in the subscriptions detail table. Toggling the classification cascades to the last 3 months of underlying `transaction_splits` rows via `reclassify_subscription_split`, upserts the override, and invalidates the detection cache atomically. The household-bills aggregate filters by effective `split_type='shared'` so personal subscriptions are correctly excluded from the household pot — and a symmetric `get_user_shared_known_bills` helper preserves flow conservation when reimbursement-mode members have personal bills.

### Bank Accounts ( `backend/modules/bank_accounts/` )

Manual bank account creation and management. Supports personal, partner, and joint (hogar) account types. Balance tracking with sync timestamps.

### Settings ( `backend/modules/settings/` )

User notification preferences, custom category ordering with drag-and-drop (up to 20 per type), category hide/show, and usage tracking. Categories fetched dynamically via API — all dropdowns across the app reflect user preferences in real time.

## Documentation

---

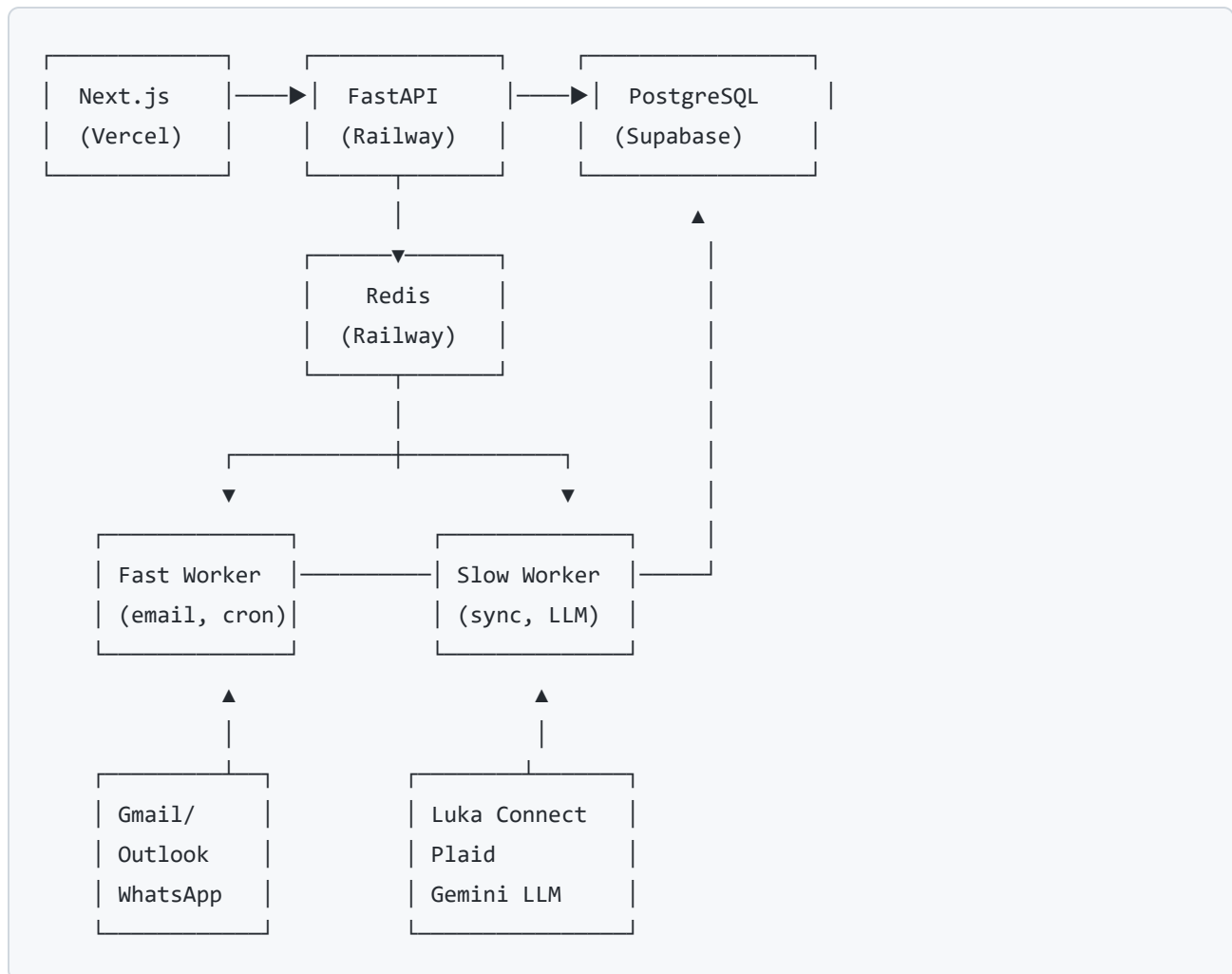
- [ARCHITECTURE.md](#) — System design, data model, API endpoints, worker jobs
- [NEXT-STEPS.md](#) — Pending work, known issues, future ideas
- [Design Specs](#) — Original design documents
- [Bank Email Research](#) — 30 banks across 6 LATAM countries

# ARCHITECTURE.md — System Architecture

## Architecture

### System Overview

Luka is a monorepo with a FastAPI backend, Next.js frontend, and two ARQ worker processes. All components share a Supabase PostgreSQL database. Redis handles caching and job queues.



# Core Components

---

## FastAPI Backend ( `backend/main.py` )

- Entry point: `main:app`
- 16 routers registered (auth, households, invite, email, whatsapp, transactions, budgets, cuota, user\_budget\_settings, bank\_accounts, settings, bank\_connect, subscriptions, notifications, merchant\_review, train, plaid)
- CacheHeaderMiddleware: `private, max-age=30` on GET endpoints
- CORS: configured from `settings.cors_origins` env var
- Health check: `GET /health`
- Talks to: PostgreSQL, Redis, Supabase Auth

## Fast Worker ( `backend/worker.py:FastWorkerSettings` )

- Handles: email processing, invite emails, cron jobs
- Config: `max_jobs=20, job_timeout=60s`
- Queue: `arq:queue` (default)
- Cron jobs: mail watch renewal (3am), raw email purge (hourly), email log purge (2am), webhook cleanup (4am), bank connect syncs (6h), subscription cache (5:30am), Plaid syncs (3:30am)

## Slow Worker ( `backend/worker.py:SlowWorkerSettings` )

- Handles: bank syncs (Connect + Plaid), LLM merchant review, reconciliation, template agent
- Config: `max_jobs=5, job_timeout=600s`
- Queue: `arq:queue:slow`
- Functions: `run_connect_sync`, `run_plaid_sync_job`, `process_merchant_review`, `run_template_agent`
- Cron: reconciliation job (6am daily), template agent (2am daily)
- Routing: jobs in `SLOW_JOBS` set in `backend/jobs/queue.py` -> slow queue

## Next.js Frontend ( `frontend/app/` )

- Framework: Next.js 16 (App Router)
- State: Zustand 5 (persisted) + TanStack Query 5
- Route groups: `(auth)`, `(dashboard)`, `(public)`
- Auth: Supabase client-side SDK + middleware redirect
- Key components in `frontend/app/(dashboard)/components/`

# Data Model

## Core Tables

| Table                                     | Purpose                                                                | Key Relationships                              |
|-------------------------------------------|------------------------------------------------------------------------|------------------------------------------------|
| <code>users</code>                        | User accounts, profile, encrypted OAuth tokens                         | -> household_members, transactions             |
| <code>households</code>                   | Shared financial unit (individual/couple)                              | -> household_members, bank_accounts, budgets   |
| <code>household_members</code>            | User<->household junction (role: owner/member)                         | -> users, households                           |
| <code>household_invites</code>            | Invitation tokens with expiry                                          | -> households                                  |
| <code>transactions</code>                 | Core financial records                                                 | -> users, households, bank_accounts, merchants |
| <code>transaction_splits</code>           | Per-tx split info (personal/partner/shared)                            | -> transactions                                |
| <code>bank_accounts</code>                | Connected accounts with sync state                                     | -> households, users                           |
| <code>merchants</code>                    | Raw merchant names -> canonical mapping                                | -> canonical_merchants                         |
| <code>canonical_merchants</code>          | Verified merchant entities                                             | <- merchants                                   |
| <code>merchant_category_selections</code> | User category choices per merchant                                     | -> merchants, users                            |
| <code>household_budgets</code>            | Monthly budget per account                                             | -> households, bank_accounts                   |
| <code>category_budgets</code>             | Per-category monthly limits                                            | -> household_budgets                           |
| <code>household_budget_allocations</code> | Legacy 50/20/30 split % (hogar/ahorro/personal)                        | -> households                                  |
| <code>user_budget_settings</code>         | Per-user savings target, payday, personal allocation amount + currency | -> users                                       |
| <code>cuota_purchases</code>              | Installment purchases (cuotas) tracked across months                   | -> users, households                           |
| <code>subscription_overrides</code>       | User overrides for detected subscriptions: status, category,           | -> users                                       |

| Table                                  | Purpose                                                                        | Key Relationships |
|----------------------------------------|--------------------------------------------------------------------------------|-------------------|
|                                        | next_charge_day, split_type<br>( <span>personal</span> / <span>shared</span> ) |                   |
| <span>bank_credentials</span>          | AES-256-GCM encrypted Luka Connect creds                                       | -> users          |
| <span>plaid_items</span>               | Plaid connection state per user                                                | -> users          |
| <span>notifications</span>             | In-app notifications                                                           | -> users          |
| <span>notification_preferences</span>  | WhatsApp toggle per user                                                       | -> users          |
| <span>user_category_preferences</span> | Custom category ordering                                                       | -> users          |
| <span>processed_webhooks</span>        | Idempotency tracking                                                           | --                |
| <span>failed_jobs</span>               | Failed job logging                                                             | --                |
| <span>merchant_review_jobs</span>      | Async LLM review batches                                                       | -> users          |

Email Pipeline Tables (migration 030)

| Table                         | Purpose                                                                                                                                       | Key Relationships  |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| <span>bank_registry</span>    | 101 banks across 6 countries (CL, CO, MX, PE, BR, US). Domain lookup, country, active template reference                                      | -> email_templates |
| <span>email_templates</span>  | Declarative JSON extraction templates. Status: candidate/active/retired/failed. Stores template_code, accuracy, validation counts             | <- bank_registry   |
| <span>parsed_email_log</span> | Audit log for every parsed email. Tracks parser used (llm/template/regex), LLM model, waterfall depth, shadow validation results. RLS enabled | -> users           |

Key Relationships

- A user belongs to one household via household\_members
- Transactions belong to a user + household + optional bank\_account + optional merchant
- Bank accounts belong to a household, linked to a user
- Budget hierarchy: household -> household\_budgets -> category\_budgets



- Bank registry links to active email templates; parsed email log tracks which parser handled each email

## Key Flows

---

### Email Transaction Flow (Three-Layer Parser)

1. Gmail/Outlook sends push notification -> `POST /webhooks/gmail` or `/webhooks/outlook`
2. Webhook handler ACKs immediately (<200ms), enqueues `process_email` to fast worker
3. Worker fetches email via Gmail API / Microsoft Graph (auto-refreshes OAuth tokens)
4. Bank registry lookup (Redis-cached): identify bank by sender domain -> get country, bank name, active template
5. Financial keyword filter: 50+ keywords in Spanish, English, Portuguese
6. **Layer 1 — Template:** If active template exists for this bank, execute declarative JSON extraction (CSS selectors + regex + fixed transforms). Zero LLM cost.
7. **Layer 2 — LLM Waterfall:** Confidence-based model escalation through 4 Gemini models:
  - gemini-3.1-flash-lite (threshold 0.9) -> gemini-2.5-flash (0.8) -> gemini-3-flash (0.7) -> gemini-2.5-pro (0.0)
  - Circuit breaker: 50% error rate over 20 requests -> 15min cooldown
  - Each model gets 2 attempts before escalating
8. **Layer 3 — Legacy Regex:** Banco de Chile format fallback with transaction type inference from subject/body keywords (person-to-person detection, CC payment detection, income keywords)
9. **Empty body guard:** Emails with no HTML/text body are skipped before parsing to prevent LLM hallucinations
10. Parsed email logged to `parsed_email_log` (parser used, model, waterfall depth, raw HTML for template training)
11. Dedup: Redis-based per-email (24h TTL) + cross-sender 5-minute window (sign-agnostic with abs() for expense amounts)
12. Merchant lookup: known -> instant category, new -> 3 LLM suggestions (Gemini)
13. Transaction saved with correct sign: expenses/transfers as negative, income as positive (unified convention across all sources)
14. WhatsApp: expenses/income get split type buttons; transfers get informational-only message (no split/category flow)

### Template Agent Flow (Daily 2am)

1. **Shadow validation:** For banks with active templates, sample 25% of recent template-parsed emails and re-parse with LLM. Any amount mismatch -> auto-retire template.

2. **Discovery:** Find banks with 20+ LLM-parsed emails but no active template.
3. **Generation:** Feed 10-20 sample emails + their LLM extractions to Gemini 2.5 Flash -> get declarative JSON template.
4. **Validation:** Execute template on all samples, compare vs LLM ground truth. Require 100% amount match, 95% merchant match.
5. **Promotion:** If passes validation, promote to active + link to bank\_registry.

## WhatsApp Conversational Flow

1. User receives transaction alert — expenses/income get interactive split buttons, transfers get informational-only message ("Ajuste entre cuentas" — no action needed)
2. User picks split type (Personal/Hogar/Pareja) for non-transfer transactions
3. Category picker presented (inline keyboard)
4. User confirms -> transaction\_split created, status updated
5. Manual expense: user sends natural language -> parsed into transaction

## Bank Sync Flow (Luka Connect)

1. User enters bank credentials -> encrypted (AES-256-GCM) -> stored in `bank_credentials`
2. `schedule_connect_syncs` cron (every 6h) enqueues `run_connect_sync` for each credential
3. Slow worker calls Luka Connect API -> scraper fetches transactions
4. Mapper deduplicates + maps to Luka transaction format -> saved to DB
5. Reconciliation job (6am daily) matches email and bank-sourced transactions

## Plaid Sync Flow

1. User connects via Plaid Link widget (production) -> public token exchanged for access token
2. `schedule_plaid_syncs` cron (3:30am daily) enqueues `run_plaid_sync_job`
3. Slow worker calls Plaid Sync API with cursor pagination
4. Smart mapper (`backend/modules/plaid/mapper.py`):
  - Extracts person names from Zelle descriptions ("Zelle payment to JOHN DOE Conf#..." → "John Doe")
  - Detects CC bill payments (Amex ACH, Chase, etc.) → only flags as "transfer" if the target card account exists in the system; otherwise treated as expense
  - Person-to-person payments (Zelle, Venmo, CashApp, PayPal) classified as expense/income, NOT transfer
  - Transaction type derived from Plaid amount sign before flip: positive = expense, negative = income
  - Amounts stored in cents (Plaid sends dollars × 100)
5. Pending (processing) Plaid transactions surfaced in frontend pending section with "bank" badge

6. Reconciliation: `find_email_match()` checks for matching pending email transactions and merges them

## Reconciliation Flow

Reconciliation runs in two places: inline during Plaid/Connect sync (immediate) and via

`run_reconciliation_job` cron (6am daily, catches stragglers).

1. For each bank tx, `find_email_match()` uses 3-tier priority matching:
  - **Exact:** same merchant (ilike), +/-2 days, exact absolute amount
  - **Fuzzy:** +/-3 days, 30% amount tolerance (absolute)
  - **Sum match:** N email txs from same merchant whose amounts sum to bank tx (5% tolerance)
2. All comparisons use `abs(amount)` — all sources now store negative for expenses, positive for income
3. On match: copies merchant\_id, category, transaction\_type from email tx -> bank tx; re-links splits; deletes email tx
4. `detect_transfers()` finds same-amount opposite-sign pairs across accounts within +/-2 days, marks both as transfer

## Encryption

- **Fernet** (`backend/core/encryption.py`): symmetric encryption for OAuth provider tokens (Google, Microsoft)
- **AES-256-GCM** (`backend/modules/bank_connect/encryption.py`): bank credentials (RUT + password), 12-byte random nonce per field

## Authentication & Authorization

- **Provider:** Supabase Auth (Google OAuth for Gmail users, Microsoft OAuth for Outlook users)
- **Token strategy:** JWT validated locally via PyJWT + JWKS (ES256 primary, RS256/HS256 fallback)
- **Frontend:** Supabase JS SDK handles token refresh; `middleware.ts` redirects unauthenticated users
- **Backend:** `get_current_user()` dependency validates JWT, caches user profile in Redis (5-min TTL)
- **Session management:** SessionGuard component — PWA persistent sessions + 30-min browser inactivity timeout with visibility change detection
- **RLS:** Supabase Row Level Security on sensitive tables (including `parsed_email_log`); SECURITY DEFINER aggregate RPC for partner data (no raw partner rows exposed)
- **OAuth tokens:** Google/Microsoft provider tokens encrypted with Fernet, stored in `users` table for API access (Gmail, Graph)

# Background Processing

## Worker Architecture

Two ARQ worker processes deployed as separate Railway services:

| Worker | Queue          | Max Jobs | Timeout | Purpose                                                |
|--------|----------------|----------|---------|--------------------------------------------------------|
| Fast   | arq:queue      | 20       | 60s     | Email processing, invite emails, cron scheduling       |
| Slow   | arq:queue:slow | 5        | 600s    | Bank syncs, LLM review, reconciliation, template agent |

Routing logic in `backend/jobs/queue.py`: jobs listed in `SLOW_JOBS` set are enqueued to slow queue.

## Cron Jobs

| Job                         | Schedule      | Worker | Purpose                                              |
|-----------------------------|---------------|--------|------------------------------------------------------|
| renew_mail_watches          | 3:00 AM daily | Fast   | Refresh Gmail watch subscriptions (7-day expiry)     |
| purge_raw_emails            | Hourly        | Fast   | Clean old email data                                 |
| purge_email_logs            | 2:00 AM daily | Fast   | Purge raw HTML from parsed_email_log after 7 days    |
| cleanup_processed_webhooks  | 4:00 AM daily | Fast   | Remove webhook dedup records                         |
| schedule_connect_syncs      | Every 6h      | Fast   | Enqueue Luka Connect sync jobs                       |
| refresh_subscriptions_cache | 5:30 AM daily | Fast   | Pre-compute recurring transaction patterns           |
| schedule_plaid_syncs        | 3:30 AM daily | Fast   | Enqueue Plaid sync jobs                              |
| run_reconciliation_job      | 6:00 AM daily | Slow   | Match email + bank transactions                      |
| run_template_agent          | 2:00 AM daily | Slow   | Shadow validate active templates + generate new ones |

Async Tasks

| Task                                 | Queue | Description                                                                                      |
|--------------------------------------|-------|--------------------------------------------------------------------------------------------------|
| <code>process_email</code>           | Fast  | Fetch email -> bank registry lookup -> three-layer parse -> categorize -> save -> WhatsApp alert |
| <code>send_invite_email</code>       | Fast  | Format and send household invite via Resend/SMTP                                                 |
| <code>run_connect_sync</code>        | Slow  | Pull transactions from Luka Connect scraper                                                      |
| <code>run_plaid_sync_job</code>      | Slow  | Sync US bank via Plaid API                                                                       |
| <code>process_merchant_review</code> | Slow  | LLM-batch group unverified merchants                                                             |
| <code>run_template_agent</code>      | Slow  | Autonomous template lifecycle management                                                         |

## External Integrations

| Service              | Purpose                                                          | Config                                                                                |
|----------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Google Cloud Pub/Sub | Gmail push notifications (real-time email alerts)                | <code>GCP_PROJECT_ID</code> , OIDC auth, topic: <code>luka-gmail-notifications</code> |
| Gmail API            | Fetch email content, manage watch subscriptions                  | OAuth tokens stored encrypted in users table                                          |
| Microsoft Graph      | Outlook email fetch + webhook subscriptions                      | <code>MICROSOFT_CLIENT_ID</code> , <code>MICROSOFT_CLIENT_SECRET</code>               |
| WhatsApp Cloud API   | Transaction alerts + conversational split flow                   | <code>WHATSAPP_PHONE_NUMBER_ID</code> , <code>WHATSAPP_ACCESS_TOKEN</code>            |
| Luka Connect         | Chilean bank scraping (Banco de Chile + 8 others)                | Separate repo, <code>LUKA_CONNECT_URL</code> + <code>LUKA_CONNECT_API_KEY</code>      |
| Plaid                | US bank account connections + transaction sync                   | <code>PLAID_CLIENT_ID</code> , <code>PLAID_SECRET</code> , <code>PLAID_ENV</code>     |
| Google Gemini        | LLM email parsing waterfall (4 models) + merchant categorization | <code>GEMINI_API_KEY</code>                                                           |
| Resend               | Transactional emails (invites) — Railway blocks SMTP             | API key in env                                                                        |
| Supabase             | PostgreSQL hosting + Auth provider + RLS                         | <code>SUPABASE_URL</code> , keys                                                      |

## Infrastructure & Deployment

### Backend (Railway)

- Builder: Dockerfile ( `backend/Dockerfile` ) — Python 3.12-slim + uv
- Start: `uvicorn main:app --host 0.0.0.0 --port $PORT`
- Workers: `python -m arq worker.FastWorkerSettings` / `worker.SlowWorkerSettings`
- Three services: API server, fast worker, slow worker
- Production URL: `https://luka-production-eb87.up.railway.app`

### Frontend (Vercel)

- Framework: Next.js (auto-detected)
- Production URL: `https://luka-lovat.vercel.app`

## Database (Supabase)

- PostgreSQL 15 with connection pooling (PgBouncer)
- `statement_cache_size=0` for asyncpg compatibility
- Connection pool: `pool_size=5`, `max_overflow=10`, `pool_recycle=3600`

## API Endpoints

---

### Auth ( `/auth` )

| Method | Path                                     | Description                                                                        |
|--------|------------------------------------------|------------------------------------------------------------------------------------|
| GET    | <code>/auth/me</code>                    | Get current user profile (incl. household contribution_mode/fixed_amount/currency) |
| PATCH  | <code>/auth/me</code>                    | Update profile (name, phone, currency); response includes contribution fields      |
| DELETE | <code>/auth/me</code>                    | Delete account (cascade)                                                           |
| POST   | <code>/auth/store-provider-tokens</code> | Store encrypted OAuth tokens                                                       |
| POST   | <code>/auth/setup-email-watch</code>     | Initialize Gmail Pub/Sub watch                                                     |
| POST   | <code>/auth/send-whatsapp-pin</code>     | Send verification PIN                                                              |
| POST   | <code>/auth/verify-whatsapp-pin</code>   | Verify PIN (5-attempt limit)                                                       |

## Transactions ( `/transactions` )

| Method | Path                                       | Description                          |
|--------|--------------------------------------------|--------------------------------------|
| GET    | <code>/transactions/mine</code>            | User's transactions (6-month window) |
| GET    | <code>/transactions/monthly-summary</code> | Monthly totals                       |
| GET    | <code>/transactions/shared</code>          | Shared household transactions        |
| GET    | <code>/transactions/pending</code>         | Pending/unreconciled transactions    |
| PATCH  | <code>/transactions/{id}/category</code>   | Update category (trans merchant)     |
| PATCH  | <code>/transactions/{id}/split-type</code> | Change split type                    |
| DELETE | <code>/transactions/{id}</code>            | Delete transaction                   |

## Households ( `/households` )

| Method    | Path                                             | Description                |
|-----------|--------------------------------------------------|----------------------------|
| POST      | <code>/households</code>                         | Create household           |
| POST      | <code>/households/{id}/invite</code>             | Generate invite link       |
| GET       | <code>/invite/{token}</code>                     | Accept invitation          |
| GET       | <code>/households/{id}/summary</code>            | Household overview         |
| GET       | <code>/households/{id}/partner-stats</code>      | Partner contribution stats |
| GET       | <code>/households/{id}/category-breakdown</code> | Spending by category       |
| GET       | <code>/households/{id}/settlement</code>         | Settlement calculation     |
| GET/PATCH | <code>/households/{id}/split-ratio</code>        | View/update split ratio    |



## Budgets ( `/budgets` )

| Method          | Path                                            | Description                                                                                                                                                                      |
|-----------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET             | <code>/budgets/v2/{household_id}</code>         | v3 multi-level Sankey response: caller-relative income breakdown, 4-level hogar / 3-level personal builders, forecast block, risk categories, runway, cuotas, savings target     |
| GET/POST        | <code>/budgets/monthly/{household_id}</code>    | Legacy monthly budget CRUD                                                                                                                                                       |
| GET             | <code>/budgets/personal/{household_id}</code>   | Legacy personal budget view                                                                                                                                                      |
| GET/POST        | <code>/budgets/allocation/{household_id}</code> | Legacy 50/20/30 allocation                                                                                                                                                       |
| GET/POST        | <code>/budgets/categories/{household_id}</code> | Per-category budgets                                                                                                                                                             |
| GET/PATCH       | <code>/budgets/settings</code>                  | User budget settings:<br>savings_target_amount,<br>savings_target_currency,<br>payday_day_of_month,<br><b>personal_allocation_amount,</b><br><b>personal_allocation_currency</b> |
| POST/GET/DELETE | <code>/cuotas</code>                            | Installment purchases (cuotas) CRUD                                                                                                                                              |

## Bank Accounts ( `/bank-accounts` )

| Method | Path                             | Description                                 |
|--------|----------------------------------|---------------------------------------------|
| GET    | <code>/bank-accounts</code>      | List connected accounts                     |
| POST   | <code>/bank-accounts</code>      | Create account manually                     |
| PATCH  | <code>/bank-accounts/{id}</code> | Update account details                      |
| DELETE | <code>/bank-accounts/{id}</code> | Delete (cascade: splits -> txns -> account) |

## Luka Connect ( `/bank-connect` )

| Method | Path                                | Description                      |
|--------|-------------------------------------|----------------------------------|
| POST   | <code>/connect</code>               | Store encrypted bank credentials |
| DELETE | <code>/disconnect</code>            | Remove bank connection           |
| POST   | <code>/sync</code>                  | Trigger manual sync              |
| GET    | <code>/sync-status</code>           | Check sync progress              |
| GET    | <code>/connections</code>           | List active connections          |
| POST   | <code>/webhooks/luka-connect</code> | Receive sync completion webhook  |

## Plaid ( `/plaid` )

| Method | Path                            | Description                      |
|--------|---------------------------------|----------------------------------|
| GET    | <code>/items</code>             | List Plaid connections           |
| POST   | <code>/create-link-token</code> | Generate Plaid Link token        |
| POST   | <code>/exchange-token</code>    | Exchange public token for access |
| DELETE | <code>/disconnect</code>        | Remove Plaid connection          |
| POST   | <code>/sync</code>              | Trigger manual Plaid sync        |

## Subscriptions ( `/subscriptions` )

| Method | Path                                 | Description                                                                                                                                                                                                                                                                            |
|--------|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/subscriptions/detected</code> | Auto-detected recurring transactions (override-merged)                                                                                                                                                                                                                                 |
| PUT    | <code>/subscriptions/override</code> | Upsert override (status/category/next_charge_day/split_type).<br>When <code>split_type</code> is set, routes through <code>reclassify_subscription_split</code> which cascades to the last 3 months of <code>transaction_splits</code> and invalidates the detection cache atomically. |
| POST   | <code>/subscriptions/refresh</code>  | Force recompute of the detection cache                                                                                                                                                                                                                                                 |

## Notifications ( `/notifications` )

| Method | Path                                     | Description         |
|--------|------------------------------------------|---------------------|
| GET    | <code>/notifications</code>              | List notifications  |
| GET    | <code>/notifications/unread-count</code> | Unread count        |
| PATCH  | <code>/notifications/{id}</code>         | Mark as read        |
| DELETE | <code>/notifications/{id}</code>         | Delete notification |

## Settings

| Method       | Path                                              | Description                    |
|--------------|---------------------------------------------------|--------------------------------|
| GET/PATCH    | <code>/notifications/preferences</code>           | Notification preferences       |
| GET/PUT/POST | <code>/categories/preferences</code>              | Category ordering + visibility |
| GET          | <code>/categories/preferences/{cat}/usage</code>  | Category usage stats           |
| POST         | <code>/categories/preferences/{cat}/delete</code> | Delete custom category         |

## Merchant Review ( `/merchant-review` )

| Method | Path                                | Description             |
|--------|-------------------------------------|-------------------------|
| GET    | <code>/job_id</code>                | Get review job          |
| GET    | <code>/job_id/status</code>         | Job progress            |
| PATCH  | <code>/job_id/merchants/{id}</code> | Update merchant mapping |
| POST   | <code>/job_id/skip</code>           | Skip review             |
| DELETE | <code>/job_id</code>                | Cancel review job       |

Merchant Training ( `/train` — local dev only)

| Method | Path                                     | Description               |
|--------|------------------------------------------|---------------------------|
| GET    | <code>/train</code>                      | Training UI page          |
| GET    | <code>/train/merchants</code>            | List unverified merchants |
| PATCH  | <code>/train/merchants/{id}</code>       | Update merchant           |
| POST   | <code>/train/merchants/{id}/merge</code> | Merge duplicates          |
| DELETE | <code>/train/merchants/{id}</code>       | Delete merchant           |
| GET    | <code>/train/stats</code>                | Merchant DB stats         |

Webhooks

| Method   | Path                            | Description                      |
|----------|---------------------------------|----------------------------------|
| POST     | <code>/webhooks/gmail</code>    | Gmail Pub/Sub push endpoint      |
| POST     | <code>/webhooks/outlook</code>  | Outlook webhook endpoint         |
| GET/POST | <code>/webhooks/whatsapp</code> | WhatsApp verification + messages |

Frontend Architecture

Framework: Next.js 16 with App Router

Route Groups:

- `(auth)` — Login, OAuth callback, onboarding (setup-household, connect-bank, verify-whatsapp), invite acceptance
- `(dashboard)` — Protected pages: home, transactions, budgets, household, subscriptions, notifications, settings
- `(public)` — Privacy policy, terms of service, data deletion (Meta compliance)

State Management:

- Zustand 5 — lightweight client state: `{ householdId, userId, userFullName, onboardingDraft }`, persisted to localStorage
- TanStack Query 5 — all API data, 30s staleTime, prefetched in `StoreInitializer`
- 8 dashboard queries prefetched on app load (no waterfall)

## Key Dashboard Components ( `frontend/app/(dashboard)/components/` ):

- `TransactionCard` — merchant, amount, direction, bank, category tag, split badge
- `RecentTransactions` — date-grouped transaction list with inline category/split editing
- `CategoryDonut` — interactive Recharts pie chart with hover tooltips
- `SpendingChart` — 6-month area chart (personal vs shared)
- `PaceChart` — daily spend vs budget pace line (on\_track green/red)
- `BalanceCard` — total checking balance across active accounts
- `BudgetBars` — horizontal progress bars color-coded by usage %
- `BudgetConfigModal` — Radix Dialog accordion mounted on `/budgets`. Five rows (Meta de ahorro · Gasto personal · Día de pago · Aporte al hogar · Topes por categoría) ported from the old `/settings` budget sections. Per-row save mutations, 900ms auto-collapse with "Guardado ✓" chip, first-open `needsSetup` nudge, mobile bottom-sheet variant. Caps editor is empty-by-default with a top-5-spent picker derived from the `budget-v2` Sankey data
- `AllocationCard` — 50/20/30 allocation sliders with suggestion pills
- `MonthSelector` — dropdown (desktop) / bottom sheet (mobile)
- `PendingBlock` — pending/unreconciled transaction buckets with date display, source badges ("email"/"bank"), correct category type by amount sign, and transfer display ("Ajuste entre cuentas")
- `MerchantCard` — merchant review approval cards (currency-aware formatting, prefetch on hover)
- `SessionGuard` — PWA persistent sessions + 30-min inactivity timeout
- `StoreInitializer` — prefetches all dashboard data on mount

## React Query Hooks ( `frontend/app/lib/hooks/` ):

- `useMyTransactions`, `useSharedTransactions`, `usePendingTransactions`, `useMonthlySpending`
- `useBudgetStatus`, `usePersonalBudget`, `useAllocation`, `useCategoryBudgets`
- `useHouseholdSummary`, `usePartnerStats`, `useCategoryBreakdown`, `useSettlement`, `useSplitRatio`
- `useNotifications`, `useUnreadCount` (30s polling)
- `useSyncStatus` (3s polling during active sync), `useBankConnections`
- `useMerchantReview`, `useReviewStatus`, `useOptimisticReview`
- `useSubscriptions`

## Key Patterns:

- Dynamic Recharts imports (~200KB deferred via `next/dynamic`)
- Optimistic updates on category changes and notification preferences
- Bottom sheets on mobile for filters and category selection
- Mobile-first responsive: floating bottom nav on mobile, sidebar on desktop
- API client ( `frontend/app/lib/api.ts` ): 50+ methods covering all backend endpoints
- Prefetch on hover for merchant review cards

## Testing

---

**Backend:** 56 test files in `backend/tests/`, ~401 tests passing, covering all major modules:

- Auth, transactions, budgets, households, email parsing, WhatsApp handler, merchant normalization
- Bank connect encryption/mapping, queue routing, worker settings, notifications, categories, subscriptions, reconciliation
- LLM parser, LLM parser integration, parser orchestrator, template executor, template agent, bank registry service
- **Budget v2/v3:** `test_budget_v2_endpoint.py` (10 tests), `test_budget_v3_sankey.py` (~28 tests covering the 4-level hogar builder, 3-level personal builder, `_pay_first_fit` routing, caller-relative privacy regression, and a parametrized flow-conservation matrix across 6 seed/view combos), `test_budget_forecast.py` (22 tests), `test_user_budget_settings.py` (9 tests), `test_contribution_modes.py` (12 tests)
- **Subscription classification:** `test_subscription_reclassify.py` (~20 tests covering the schema, upsert, 3-month cascade, override-wins, household/personal bill filters, and router endpoint integration)
- Config: `asyncio_mode = "auto"`, `testpaths = ["tests"]`
- Run: `cd backend && pytest -v`

**Frontend:** No test infrastructure configured (no Jest/Vitest/Playwright)

**CI/CD:** No GitHub Actions pipeline. Deployment is manual via Railway (backend) and Vercel (frontend auto-deploy on push)

## Design Decisions

---

**Three-layer email parser:** Templates are cheapest (zero LLM cost), LLM waterfall handles unknown formats, regex catches legacy patterns. Template Agent autonomously upgrades banks from Layer 2 to Layer 1 as data accumulates.

**Confidence-based LLM waterfall:** Cheaper models try first; only escalate to expensive models when confidence is low. Each step has a threshold (0.9/0.8/0.7/0.0). This minimizes cost while maintaining accuracy.

**Circuit breaker on Gemini API:** If 50%+ of API calls fail in a 20-request window, skip LLM for 15 minutes and fall back to regex. Prevents cascade failures.

**DB-backed bank registry over hardcoded domains:** 101 banks across 6 countries need structured metadata (country, known subjects, active templates). Redis cache (24h TTL) keeps lookups fast.

**Two-tier worker queue:** Email webhooks must ACK in <200ms, but bank syncs can take 10+ minutes. Splitting into fast (`max_jobs=20`, 60s) and slow (`max_jobs=5`, 600s) workers prevents slow jobs from blocking webhook processing.

**No debt ledger:** Partner contributions are computed via aggregate queries on transactions, not maintained in a separate table. This avoids dual-write consistency issues and keeps the schema simpler.

**Global merchant DB:** Merchant categorizations are shared across all users to build a training dataset. Individual users can override categories via `merchant_category_selections`.

**PyJWT over Supabase SDK for auth:** Local JWT validation with JWKS avoids a network round-trip to Supabase on every request. Fallback chain (ES256 -> RS256 -> HS256) handles Supabase key rotation.

**Redis user cache:** `get_current_user()` caches user profiles for 5 minutes, avoiding repeated DB lookups on sequential API calls from the same user.

**Joint account auto-classification:** Bank accounts with `account_type = 'joint'` auto-classify all transactions as shared, skipping the WhatsApp split question.

**Luka Connect over Fintoc:** Fintoc was removed entirely and replaced with Luka Connect (standalone bank scraper). Luka Connect supports 9 Chilean banks via browser automation, stored as AES-256-GCM encrypted credentials.

**Unified amount sign convention:** All transaction sources (email, Plaid, bank connect) store expenses/transfers as negative amounts and income as positive. Email pipeline infers `transaction_type` from LLM/regex output and flips sign accordingly. This eliminates sign-mismatch bugs in reconciliation, frontend display, and dedup logic. A data migration converted existing email transactions to negative expense amounts.

**Transaction type classification (expense vs transfer vs income):** Person-to-person payments (Zelle, bank transfers to other people) are classified as expense or income, NOT transfer. "Transfer" is reserved for own-account moves: CC bill payments, checking-to-savings, ATM cash. This distinction drives the WhatsApp flow (transfers get informational-only messages) and frontend display (transfers show "Ajuste entre cuentas" instead of category/split controls).

**3-tier reconciliation:** Email transactions arrive first (real-time), bank transactions arrive later (scheduled sync). Reconciliation matches them with decreasing confidence: exact -> fuzzy -> sum-match. Email data enriches bank transactions (user-edited categories transfer over), then email duplicates are deleted.

**Per-user merchant overrides:** `merchant_category_selections` supports both global (shared dataset) and per-user category choices via nullable `user_id` FK. Users see their own overrides first, global selections second, LLM suggestions last.

**Subscription detection with explicit classification override:** Recurring expenses are detected from transaction patterns (2+ consecutive months, +/-20% amount tolerance), cached in `detected_subscriptions_cache`, and refreshed on demand. Each detected subscription's `split_type` defaults to the inferred value from the underlying `transaction_splits` row, but users can explicitly override via a click-to-flip pill on the subscriptions page. The override is stored on `subscription_overrides.split_type` and cascades atomically to the last 3 months of `transaction_splits` via `reclassify_subscription_split` — so the next time the detection runs, the

new classification is already reflected in the underlying data. The household-bills aggregate filters by effective shared-only, with a symmetric `get_user_shared_known_bills` helper used in `_reimbursement_members_known_bills` so flow conservation holds when reimbursement-mode members have personal bills.

**v3 Sankey: caller-relative privacy by construction:** The household Sankey shows the viewer's own income categories broken out at Level 0, while other household members appear as exactly one aggregated node per member ( `Ingresos {Name}` for full-mode members, `Contribución fija {Name}` for fixed-mode members). The privacy invariant — "fixed-mode members' real income is never read" — is enforced structurally in `contribution_service.income_breakdown_for_household_view`: the dispatch on `contribution_mode` happens before any non-caller transaction read, and the `fixed` branch reads only the `fixed_contribution_amount` from a single member-table SELECT result row, never querying the member's `transactions` table. This is verified end-to-end by `test_hogar_fixed_privacy_recursive_walk`, which seeds a synthetic forbidden value as a fixed-mode member's real income and walks every node and link in the response JSON to confirm the value never leaks.

**v3 Sankey: hub-based flow routing:** Both the 4-level hogar builder and the 3-level personal builder collapse all sources through a single `ingresos_hogar` / `ingresos_personales` hub node at Level 1. This makes flow conservation trivial (every source emits one link to the hub; the hub emits one link per allocation; every allocation either terminates or fans out to a single breakdown level). When income can't cover all allocations, a synthetic `otras_fuentes` source enters at Level 0 alongside the real sources, absorbed via the `_pay_first_fit` routing primitive. Verified across 6 seeded household + view combos by `TestFlowConservationAllSeeds`.



# NEXT-STEPS.md — Pending Work & Roadmap

## Next Steps

Last updated: 2026-04-16

## Recently Shipped

- Budget config modal on /budgets** — A single accordion modal triggered by a gear button next to the currency toggle on `/budgets`, consolidating the 5 budget knobs (savings target, payday, personal allocation, contribution mode, per-category caps) that used to live scattered across `/settings`. The category-caps editor is redesigned: empty-by-default list of active caps + a `+ Agregar tope` picker that surfaces the top-5 spent categories from the current month's `budget-v2` Sankey data (no more 20 empty rows). Each row has its own per-row save mutation with a 900ms auto-collapse and a "Guardado ✓" chip. Mobile becomes a bottom sheet via the existing `animate-slide-up` keyframe. Backend shim: `/auth/me` now also returns the caller's `contribution_mode` / `fixed_contribution_amount` / `fixed_contribution_currency` fields (PATCH and GET both populate them) — resolves a pre-existing TODO where the old `ContributionSection` always defaulted to `mode='full'` on mount because there was no GET path for the value. Old `BudgetSettingsSection`, `ContributionSection`, `CategoryBudgetsSection` deleted. Spec: `docs/superpowers/specs/2026-04-15-budget-inline-config-design.md`. Plan: `docs/superpowers/plans/2026-04-15-budget-inline-config-plan.md`.
- Budget v3 Sankey redesign** — Two-plan sprint (subscription classification prerequisite + v3 Sankey redesign). New 4-level hogar Sankey (per-source income → `Ingresos Hogar` hub → 5 allocation nodes including new `Gasto personal` → per-category breakdown) and 3-level personal Sankey with its own `Mis ingresos` hub. **Caller-relative privacy**: each viewer sees their own income categories broken out at Level 0 while other members appear as one aggregated node per member. Privacy invariant enforced by construction in `contribution_service.income_breakdown_for_household_view`. New `_build_hogar_sankey` / `_build_personal_sankey` builders share a `_pay_first_fit` routing primitive; legacy `_build_sankey` deleted. Frontend `BudgetSankey.tsx` rewritten with rank-based label placement reading new `level` / `kind` / `member_id` fields. New `personal_allocation_amount` user setting with input on `/settings`. Migrations 036 (`subscription_overrides.split_type`) and 037 (`user_budget_settings.personal_allocation_amount`). 401 backend tests pass; 28 new tests in

`test_budget_v3_sankey.py` covering caller-relative privacy regression, recursive-walk forbidden-value detection, and parametrized flow conservation across all seeded households + currencies. Specs: `docs/superpowers/specs/2026-04-15-budget-v3-sankey-redesign-design.md`. Plans: `docs/superpowers/plans/2026-04-15-budget-v3-subscription-classification-plan.md` and `docs/superpowers/plans/2026-04-15-budget-v3-sankey-redesign-plan.md`. Completion checkpoints: `docs/superpowers/specs/reviews/2026-04-15-budget-v3-prerequisite-completion.md` and `docs/superpowers/specs/reviews/2026-04-15-budget-v3-completion.md`.

- **Subscription classification toggle (Plan 1 prerequisite)** — Users can now click a `Personal` / `Compartido` pill on each detected subscription in `/subscriptions` to explicitly classify it as personal or shared. The toggle cascades atomically to the last 3 months of underlying `transaction_splits` rows via `reclassify_subscription_split`, upserts the override on `subscription_overrides`, and invalidates the detection cache. The household-bills aggregate now filters by effective `split_type='shared'` so personal subscriptions are correctly excluded from the household pot. A symmetric `get_user_shared_known_bills` helper is used by `_reimbursement_members_known_bills` so flow conservation holds when reimbursement-mode members have personal bills.
- **Budget page redesign (budget-v2)** — The 1-week parallel sprint (2026-04-14) shipped the original `/budgets/v2` endpoint, contribution-mode-aware privacy (full / fija / sólo reembolso), cuotas manual entry, savings target + payday day configuration, per-category budget caps, silent-by-default risk alert band, runway card with days-to-payday, and overspent-month `otras_fuentes` visualization. **Superseded by the v3 Sankey redesign above** for the visualization layer, but the underlying data model (cuotas, user\_budget\_settings, contribution\_mode dispatch) remains live.

## In Progress

---

- **Template Agent production monitoring** — Template agent deployed (2am daily cron on slow worker). Monitoring first autonomous template generations and shadow validations. First LLM-parsed transactions confirmed live (BofA via gemini-2.5-flash, 0.98 confidence).
- **Plaid production rollout** — Plaid production credentials configured and deployed. BofA connected successfully, 290 transactions synced. Zelle person extraction and CC payment detection working. Pending (processing) transactions now visible in frontend. OAuth redirect URIs configured.

## Pending

---

### Budget v3 follow-ups (deferred from review notes — non-blocking)

These items were caught by code-quality reviewers during the Plan 1 + Plan 2 sprints and explicitly deferred to keep velocity. None block any current functionality.

- **Migration constraint naming inconsistency** (Plan 1 Task 1 review): `ck_subscription_overrides_split_type` (plural) vs the existing `ck_subscription_override_status` / `_day` (singular) on the same table. Cosmetic; consider a follow-up migration that renames the older two for consistency, or accept the new plural form as canonical.
- **Race condition in `reclassify_subscription_split` cascade** (Plan 1 Task 4 review): the per-tx UPDATE-then-INSERT loop can create duplicate `transaction_splits` rows if two requests race for a tx that has no existing split row. Schema doesn't enforce uniqueness on `transaction_splits.transaction_id`. Recommended fix: add a UNIQUE constraint migration on `transaction_splits.transaction_id` and convert the loop to `INSERT ... ON CONFLICT (transaction_id) DO UPDATE`. Also benefits 4 other insertion sites in the codebase.
- **Test fixture model registration gap** (Plan 1 Task 4 review): `noqa: F401` model imports are sprinkled across `test_subscription_reclassify.py`, `test_user_budget_settings.py`, `main.py`, `alembic/env.py`, and `tests/conftest.py` to register FK targets in SQLAlchemy metadata. The right fix is a central `backend/modules/__init__.py` (or similar) that imports all models once.
- **Test spacing in cascade tests** (Plan 1 Task 4 review): `test_subscription_reclassify.py::test_cascade_updates_last_3_months_only` uses `timedelta(days=31 * months_ago)` instead of `dateutil.relativedelta(months=N)`. Works correctly with an inline comment, but `relativedelta` would be more idiomatic. Trivial cleanup.
- **Helper duplication across test files** (Plan 1 Task 5 review): `_get_seed_user`, `_get_seed_household_id`, `_user_by_email` are now defined in 4+ test files with slight naming variations. A shared `backend/tests/helpers/seed.py` would consolidate them.
- **v2 personal Sankey shared-bills double-count** (Plan 1 Task 6 review): `v2_service.py:553` calls `get_user_known_bills` (all bills) for the personal view of the legacy `_build_sankey` path. The v3 personal Sankey uses `get_user_personal_known_bills` instead, so the bug ages out as v3 adoption is confirmed; if the legacy path is removed, this can be deleted.
- **`get_budget_v2` size growth** (Plan 2 Task 9 review): the function grew significantly with the inline personal-view bucketing logic. Could be extracted to a private helper if the function gets more complex in the future.
- **`update_personal_allocation` and `update_savings_target` parallel implementations** (Plan 2 Task 11 review): could share a single `update_user_budget_settings_field` helper to remove the parallel-implementation tax — minor refactor.
- **Hogar Sankey allocation-node `if` blocks repetition** (Plan 2 Task 7 review): the 5 `if X > _ZERO:` `nodes.append(SankeyNode(...))` stanzas could be DRY'd via a list-of-tuples loop. Cosmetic.
- **Missing `otras_fuentes` synthetic-source unit test** (Plan 2 Task 7 review): `test_budget_v3_sankey.py` doesn't directly test the overspent-month `otras_fuentes` emission path. The logic is correct by inspection and integration-tested via the flow conservation matrix, but a focused unit test would harden coverage.

## Email Pipeline

- **WhatsApp message templates** — Need to create Meta-approved WhatsApp message templates (`verification_code`, `transaction_alert`) for 24-hour window bypass. Currently limited to 24h response window.
- **Income-aware WhatsApp flow** — LLM + split flow not yet income-aware for inflows. WhatsApp alerts fire for income transactions but the conversational flow assumes expenses.

## Outlook Integration

- **Microsoft Azure / Outlook push notifications** — Azure app registration needed. Requires: `Mail.Read` delegated permission, redirect URI, `MICROSOFT_CLIENT_ID`, `MICROSOFT_CLIENT_SECRET`, `OUTLOOK_CLIENT_STATE`. Code exists (`OutlookProvider`) but credentials not configured.

## Merchant System

- **Merchant training curation** — Use `/train` UI to verify, merge, and fix merchant categories. Initial seed: 161 canonical merchants from 229 raw names. Ongoing quality improvement.

## LATAM Expansion

- **Collect real email samples** — Bank registry has 101 banks seeded but most lack real email samples for template generation. Priority: Chilean banks (Banco Falabella, BCI, Banco Estado, Banco Itau, Scotiabank), then Colombian and Mexican banks.
- **Template Agent tuning** — Adjust `TEMPLATE_AGENT_MIN_EMAILS` (currently 20) and `TEMPLATE_AGENT_RECENTY_DAYS` (14) based on real-world email volume per bank.

## Multi-Card Reconciliation

- **Cross-account transfer matching** — When a user connects a second card (e.g. Amex), existing CC payment transactions should auto-reclassify from "expense" to "transfer" and reconcile against the card-side entries.

## Phase 2 Forecast Engine (carried from budget-v2 sprint)

- **Phase 2 forecast engine** — Swap `forecast.py` heuristic internals for a Bayesian engine (same function signatures; contract is stable per the 2026-04-14 spec §6.2). Adds uncertainty bands to the risk alert band and better overshoot probabilities for low-history categories.
- **Redis caching for risk categories** — The 2026-04-14 spec §6.3 mentions `budget:risk:{user_id}:{YYYY-MM}` cache with month-long TTL. Deferred in sprint; `v2_service.py` has a `TODO(Phase 2)` marker at the risk-computation block.

## Known Issues

---

- **No browser-based UAT for the v3 Sankey** — Google OAuth's automation-detection blocks chrome-devtools-mcp + browser-use from signing into Luka. The v3 Sankey redesign was verified via TypeScript build + 401 backend tests + code-level audit. Manual click-through verification by the maintainer is recommended before any major release tagging.
- **WhatsApp Business API approval** — Currently using test/dev mode. For multi-user production use, Meta's official WhatsApp Business API approval process is needed.
- **No frontend test infrastructure** — No Jest, Vitest, or Playwright configured. Backend has 56 test files (~401 tests) but frontend has zero tests.
- **No CI/CD pipeline** — No GitHub Actions workflows. Backend tests are run manually ( `pytest` ). Deployment is manual via Railway/Vercel.
- **Personal view `disponible_personal` semantic limitation** — For a full-mode caller, the personal view's `disponible_personal` appears large because the Sankey doesn't subtract the caller's implicit household contribution. Documented in the v3 spec §9 as a known limitation; the clean fix would be either an `aporte_hogar` Level-1 outflow node or capping the personal view's income at `personal_allocation_amount` when set. Neither was shipped in v3 — see the spec for the rationale.

## Infrastructure TODOs

---

- **Luka Connect env vars** — Verify `LUKA_CONNECT_URL`, `LUKA_CONNECT_API_KEY`, `CONNECT_ENCRYPTION_KEY`, `BACKEND_PUBLIC_URL` are set on production Railway.
- **Supabase OAuth providers** — Verify Google OAuth and Microsoft OAuth are enabled in Supabase Auth -> Providers.

## Future Ideas

---

- Category budget alerts via WhatsApp (notify when spending approaches category limit)
- Template Agent dashboard — admin UI to view template status, accuracy, shadow validation results per bank
- Multi-currency budget support (currently assumes single currency per household)
- Bank registry admin API — CRUD endpoints for managing bank\_registry entries

- Per-transaction `is_shared` override — a user might want to override a one-off transaction without touching the subscription rule (the v3 sprint considered this and explicitly deferred it; out of scope for v3)